

Educational Codeforces Round 188知识点总结

前言

这份讲义不是把每道题的代码抄一遍，而是想把这场比赛里真正有价值的思考方式沉淀下来。

做完一场比赛之后，最值得复盘的通常不是“我写了什么代码”，而是下面这些问题：

- 我为什么会想到这个方向？
- 题目真正的本质是什么？
- 有没有一个关键观察，一旦看出来，整题就简单了？
- 以后再遇到类似题，我应该联想到什么模型？

如果你带着这些问题读这份讲义，那么它更像是一份赛后课程总结，而不是普通笔记。

这场比赛的整体结构很适合训练以下能力：

1. 从过程里提炼规律，而不是盲目模拟。
2. 把自然语言翻译成数学或图论模型。
3. 从复杂状态中提炼出真正需要维护的核心量。
4. 遇到大步数线性转移时，敏感地想到矩阵快速幂。

本场比赛的整体分层

从训练角度看，这场题大致可以分成三层。

第一层：A-B

这两题非常适合练“观察能力”。

它们都给了一个过程，看起来像模拟题，但真正解法都不是老老实实地去模拟很多轮，而是先抓住决定答案的那个关键位置。

如果你做这种题时仍然习惯“走一步看一步”，说明你还没有把注意力放在“本质结构”上。

第二层：C-D-E

这三题开始进入标准建模阶段。

- C 是周期、倍数、lcm 和容斥。
- D 是交替性质转二分图。
- E 是构造题，要求你同时处理“数值过程”和“字符频数”。

这三题共同训练的，是“把题目语言翻译成熟悉模型”的能力。

第三层：F-G

这两题是本场最值得反复咀嚼的部分。

- **F** 训练你把局部操作问题，转成极值贡献问题。
- **G** 训练你从网格过程抽象出区间状态，再继续压缩成线性转移。

如果你能真正吃透这两题，说明你的抽象能力已经开始往更高层走了。

A. Passing the Ball

一、题目本质

这题表面上是在传球，实际上是在问：

球最远能扩展到哪里？

很多同学第一次看到这题，会自然地想到：

- 开一个数组记录哪些点访问过；
- 模拟传球过程；
- 如果出现环，就继续直到发现不再访问新点。

这些想法不能说完全错，但它们都没有抓住这题真正简单的地方。

这题真正关键的是：

球什么时候第一次不能继续向右扩展？

只要回答了这个问题，答案就出来了。

二、为什么答案是“第一个 **L** 的位置”

题目保证：

- 第一个字符是 **R**
- 最后一个字符是 **L**

于是球从 **1** 出发后，第一步一定会到 **2**。

接下来想象球第一次到达某个位置 **i**：

- 如果 $s[i] = \text{R}$ ，那它还能继续向右走；
- 如果 $s[i] = \text{L}$ ，那它会立刻往左回去。

所以结论非常自然：

- 只要还没遇到 **L**，球就还能继续往右扩展；
- 一旦第一次遇到 **L**，球就会开始向左弹回，不可能再把右边的新点访问到。

因此，所有被访问到的点一定构成一个前缀：

$$1, 2, 3, \dots, pos$$

其中 **pos** 就是字符串中第一个 **L** 出现的位置。

三、这题真正想训练你的是什么

1. 不要一看到过程就开始模拟

很多题都会故意给你一个“动态过程”，但竞赛中你要养成一个习惯：

先别急着模拟，先问：答案到底是由哪个关键量决定的？

这题里真正决定答案的，不是球一共传了多少轮，而是：

- 第一次改变方向的位置在哪里。

2. “第一次出问题的位置”是观察题的高频答案

以后看到类似题时，你可以优先检查：

- 第一个不满足条件的位置；
- 第一个阻止继续扩展的位置；
- 第一次发生行为变化的位置。

这类“第一次”往往比全过程更重要。

四、易错点

1. 把这题当成环上模拟题。
2. 忘记题目问的是“被访问过的点数”，不是最后停在哪。
3. 没有利用首字符是 `R`、末字符是 `L` 这个强条件。

B. Right Maximum

一、题目本质

每次操作：

1. 找最大值；
2. 如果最大值有多个，取最右边那个；
3. 删除它以及它右边所有元素。

问总共会做多少次。

很多同学的第一反应是：

- 每次重新找最大值；
- 再删掉一个后缀；
- 直到数组为空。

这当然能描述过程，但仍然没有把题目“压扁”。

真正应该想的是：

每轮操作结束后，数组会变成什么样？

答案是：

- 每次删完以后，剩下的一定还是一个前缀。

于是整个问题，就变成了一个“前缀不断缩短”的过程。

二、为什么会变成“不断跳前缀”

设 `last[i]` 表示：

- 在前缀 `a[1..i]` 中，
- 最大值最后一次出现的位置。

也就是说，它是“前缀最右最大值”。

那么对于当前前缀 `1..i`：

- 这一轮会选 `last[i]`；
- 然后删除 `last[i]..i`；
- 下一轮只剩下前缀 `1..last[i]-1`。

所以整个过程就是：

- 从 `i = n` 开始；
- 跳到 `last[i] - 1`；
- 再跳；
- 再跳；
- 直到跳到 `0`。

每跳一次，就是一次操作。

这个视角一旦建立起来，题目就从“模拟删除”变成了“若干次跳跃计数”。

三、这题教你的思维方式

1. 删除类问题，先看“删完剩什么”

很多人做删除题，注意力总放在“删掉了什么”。

但竞赛里更重要的，往往是：

- 删完之后，剩下的结构是什么？

这题中，删完之后剩的始终是一个前缀，所以就有了非常规整的跳跃过程。

2. “最右最大值”不是细节，而是题眼

如果题目没有强调“最右”，这题结构就会完全不同。

所以以后看到“最左”“最右”“第一次”“最后一次”这类词时，不要把它们当成普通细节，它们经常就是整个建模的核心。

四、易错点

1. 每轮都重新扫描最大值，虽然能做，但没抓住结构。
 2. 忘了是“最右最大值”。
 3. 没看出删完后始终剩前缀。
-

C. Spring

一、题目本质

三个人分别按周期来打水：

- Alice 每 a 天来一次；
- Bob 每 b 天来一次；
- Carol 每 c 天来一次。

然后根据同时来的人数不同，分水量也不同。

这题的本质是三件事：

1. 倍数统计；
2. 公倍数统计；
3. 容斥思想。

这是竞赛里最标准的一类基础数论题之一。

二、看到“周期发生”时，第一反应应该是什么

如果某个事件每 x 天发生一次，那么它发生的时间一定是：

- x
- $2x$
- $3x$
- ...

于是，在 $1..m$ 中出现的次数就是：

$$\left\lfloor \frac{m}{x} \right\rfloor$$

这一步看起来基础，但非常重要，因为它是周期题最基本的翻译方式。

以后你只要看到：

- 每隔若干天发生一次；
- 每逢某个周期发生一次；
- 若干周期事件重叠；

都应该先想到“倍数个数”。

三、看到“同时发生”时，第一反应应该是什么

如果两个人在同一天来，那么这一天必须同时满足：

- 是 a 的倍数；
- 是 b 的倍数。

这就说明，这一天必须是：

$$lcm(a, b)$$

的倍数。

所以两个人同时来访的次数就是：

$$\left\lfloor \frac{m}{lcm(a, b)} \right\rfloor$$

三个人同理：

$$\left\lfloor \frac{m}{lcm(a, b, c)} \right\rfloor$$

这一步一定要形成反射：

“同时满足多个周期条件”
等价于
“是这些周期的最小公倍数的倍数”。

四、为什么这题不用死背容斥公式

很多同学一看到这种题，就开始想：

- 单人集合；
- 两两交集；
- 三重交集；
- 完整容斥。

这当然没错，但这题更自然的做法其实是：

直接按每个人的贡献来拆。

例如对 Alice 而言：

- 她单独来时拿 6 ；
- 她和另一个人同来时拿 3 ；
- 三个人同来时拿 2 。

所以你要做的不是背公式，而是先问：

每种场景下，这个人到底贡献了多少？

然后再去数：

- 她总共来了多少次；
- 她和别人两两重合了多少次；

- 三个人一起出现了多少次。

这类题最怕的不是不会算，而是没把“场景”和“贡献”对应清楚。

五、这题真正应该带走的知识点

1. 周期题先数倍数

这是第一反应。

2. 共同发生先想 1cm

这是第二反应。

3. 容斥不一定要写成死板集合形式

有时候“按贡献拆”更自然、更不容易乱。

六、易错点

1. $1\text{cm}(a, b)$ 写成 $a * b / \text{gcd}(a, b)$ 容易溢出。

更稳的写法是：

- $a / \text{gcd}(a, b) * b$

2. 三个数的 1cm 忘了分步做。
3. 没有想清楚不同重合场景下每个人到底拿多少。

D. Alternating Path

一、题目本质

这题读起来很绕：

- 要给无向图定向；
- 定义什么叫 alternating path；
- 再定义哪些点是 beautiful。

如果你一直跟着定义本身走，就很容易越读越乱。

这题真正的核心是：

题目要求一种“全局一致的交替性”。

而一旦你在图论里看到：

- 相邻关系要交替；
- 奇偶层要统一；
- 不同层的角色不同；

你就应该立刻想到：

- 这会不会是二分图？

这就是这题最重要的建模入口。

二、为什么这里会想到二分图

很多同学觉得“二分图”必须长得很明显，比如题目直接说：

- 相邻点不能同色；
- 图能否黑白染色。

但在竞赛里，题目经常不会直接这么说，它只会给你一个包装过的语言。

这题里的包装词包括：

- 交替；
- 路径方向前后交替；
- 从一个点出发的任意路径都要满足这种规律。

这些词虽然表面上在讲路径，但本质上都在暗示一件事：

- 点之间必须被分成两类；
- 相邻点分属不同类；
- 奇偶层角色不同。

这就是二分图。

所以这题最值得学的，不是 BFS 染色本身，而是：

以后题目只要出现“交替”“奇偶层”“相邻关系必须不同”，就要对二分图高度敏感。

三、标准做法为什么合理

对于每个连通块：

1. 做 BFS 或 DFS 染色；
2. 相邻点染成相反颜色；
3. 如果发现相邻点颜色相同，则这个块不是二分图。

如果一个连通块是二分图，那么它天然分成两边：

- `cnt[0]`
- `cnt[1]`

再根据题目含义，决定合法点集应该取哪一边。

这题最后是取：

$$\max(cnt[0], cnt[1])$$

因为你可以选择更大的一边作为满足条件的集合。

四、这题真正想训练你的能力

1. 会把题面语言翻译成熟悉模型

图论题很多时候难不在算法，而在于：

- 你能不能把原题语言翻译成“这其实是在问二分图”。

这一步如果不会，后面再熟悉 BFS 也没用。

2. 学会按连通块独立处理

很多图论题不是整个图统一算，而是：

- 每个连通块单独判断；
- 每个连通块单独贡献；
- 最后把贡献加起来。

这也是图论里很常见的结构。

五、易错点

1. 只判断整张图是否二分，而忘了按连通块累计答案。
 2. 染色时没有正确处理已经染色的邻点。
 3. 没有把题面中的“交替性”抽象成奇偶层关系。
-

E. Sum of Digits (and Again)

一、题目本质

题目给你一串数字字符，可以任意重排。

要你把它重排成某个 $S(x)$ 。

其中：

- 先写 x
- 再写 $\text{sumdigits}(x)$
- 再写 $\text{sumdigits}(\text{sumdigits}(x))$
- ...
- 直到变成一位数为止

这题本质上同时考察两层东西：

1. 数值过程；
2. 字符 multiset。

如果你把这两层混在一起想，就很容易乱。

二、第一层：先看数值过程本身

设 $f(x)$ 表示 x 的数位和。

那么题目中的串就是：

$$S(x) = x \parallel f(x) \parallel f(f(x)) \parallel \dots$$

其中 $\parallel$ 表示字符串拼接。

这类过程最重要的一个性质是：

- 它会快速缩小。

例如：

- 75 -> 12 -> 3
- 99999 -> 45 -> 9

也就是说，虽然最开始的 x 可能不小，但后面的链很短。

这个观察很重要，因为它告诉你：

后缀过程虽然看起来像递归，但实际很短，很适合枚举和构造。

三、第二层：顺序不重要，频数才重要

因为题目允许重排，所以原串的顺序根本不重要。

真正重要的是：

- 0 出现了几次；
- 1 出现了几次；
- ...
- 9 出现了几次。

这说明本题本质上不是“字符串匹配”，而是：

- 字符频数匹配
- 或者说 multiset 匹配

所以以后你只要看到“可以重排”“打乱顺序”“任意排列”，第一反应就应该是：

顺序不是重点，频数才是重点。

四、这题为什么不适合直接枚举原始 x

很多同学的第一想法是：

- 枚举 x ；
- 生成 $S(x)$ ；
- 看和原串是否匹配。

问题在于：

- 真正的 x 可能很大；
- 没有明显的范围可直接枚举。

所以更自然的思路是：

1. 枚举链中的某个关键值；
2. 先生成后缀链；
3. 看剩余字符能不能补成合法前缀。

这就是典型的构造题思路：

- 先固定一部分结构
- 再用剩余资源补另一部分

这种思维以后做构造题非常常见。

五、为什么这题容易出错

1. 一位数特判很容易漏

如果最后答案本身就是一位数，那么整个 $S(x)$ 其实就只有那一位。

这种边界情况特别容易在构造时被漏掉，因为很多写法默认答案会拆成：

- 前缀；
- 后缀链。

但一位数时，根本没有“后缀链”的扩展意义。

2. 只看数位和，不看频数

这是构造题中非常典型的误区。

例如两个串数位和相同，并不代表它们的字符 multiset 相同。

所以这类题一定要按频数检查，而不是只按和检查。

3. 前导零问题

只要你在构造“前缀数字”，就必须警惕：

- 首位能不能是 0？

这类问题在构造题里极高频。

六、这题真正想训练你的东西

1. 会把“数值过程”和“字符过程”分开看。
 2. 看到“可重排”时，主动转到频数视角。
 3. 会做“先定后缀，再补前缀”的反向构造。
-

七、以后遇到类似题时怎么想

建议按这个顺序问自己：

1. 过程本身会不会快速收缩？
2. 顺序是不是不重要？
3. 能不能先固定一部分，再补另一部分？
4. 需要检查哪些合法性条件？

如果这四个问题你都习惯性去想，那么构造题会越来越稳。

F. Sum of Fractions

一、这题为什么非常值得复习

这题是这场最有代表性的综合题之一。

它不是那种只靠一个模板秒掉的题，而是把下面几个经典思想串在了一起：

- 先研究单个对象；
- 再比较多对象之间的边际收益；
- 证明操作只会落在极值上；
- 然后转成子数组极值统计；
- 最后再配合单调栈、离线、逆元做完。

如果你能把这题真正理解透，以后很多综合题都会更顺。

二、第一层：为什么一定要先研究单个分数

这类题最怕一上来就想整体。

正确的习惯应该是：

先把最小单位研究清楚，再去看整体怎么拼。

这题的最小单位就是单个分数。

你应该先问：

- 一个分数做若干次操作后，最优结果是什么？
- 每一步操作的收益是多少？
- 收益序列长什么样？

只有这一步清楚了，后面你才能比较：

- 多个分数之间，操作应该给谁。

这其实是很多综合题的通用思路：

1. 先研究一个；
2. 再研究多个；
3. 再研究所有区间或所有结构的总和。

三、第二层：为什么要比较边际收益

这题最核心的思想，不是“最终谁更大”，而是：

- 当前这一步，给谁操作最赚？

这就是所谓的 **边际收益**。

做贪心题时，一个非常重要的思维方式是：

不要只比较最终结果，要比较“下一步收益”。

如果某个对象在任意时刻的下一步收益都不比别的对象差，那么所有操作都应该优先给它。

这就是贪心正确性的来源。

在这题里，最后会发现：

- 分母越小，下一步收益越不差。

所以对于一个子数组而言：

- 最优策略就是一直操作最小值对应的那个分数。

这是全题最关键的一步。

四、第三层：为什么会转成“子数组最小值问题”

一旦你证明了：

- 对每个子数组，所有操作都会给最小值对应分数，

那么整个问题立刻发生质变。

原本你还在想：

- 子数组里有很多分数；
- 操作可以分配给不同位置；
- 可能有复杂互动。

现在这些都不重要了。

你真正需要关心的变成：

1. 子数组的基础贡献；
2. 子数组最小值的额外贡献。

这一步是竞赛中的一个非常经典转换：

一旦答案只和子数组极值有关，就要往“子数组极值贡献统计”上靠。

这类题以后你一旦闻到“最值决定一切”的味道，就应该立刻想到单调栈。

五、第四层：为什么单调栈会出现

现在你要统计的是：

- 每个位置作为子数组最小值时，能负责多少个子数组。

这正是单调栈最经典的职责之一。

设：

- $L[i]$ 是左边第一个更小的位置；
- $R[i]$ 是右边第一个更小的位置。

那么位置 i 作为某个代表元时，能负责的子数组数量就是：

$$(i - L[i]) \times (R[i] - i)$$

这公式不要死背，最好理解它的含义：

- 左边起点有多少种选法；
- 右边终点有多少种选法；
- 两边独立，所以乘起来。

只要你真的理解了“左右独立选择”，这个公式以后就不会忘。

六、第五层：为什么还要“按值合并”

这题比普通单调栈题更进一步。

单调栈只能告诉你：

- 某个位置贡献了多少次。

但本题的后续查询并不关心“这个位置是谁”，而只关心：

- 最小值是多少。

所以你要继续做一步：

- 把所有最小值相同的位置贡献合并起来。

也就是得到：

- $cnt[x] = \text{最小值等于 } x \text{ 的子数组个数}$

这一步非常值得记，因为很多题都是这样：

1. 先按位置统计；
2. 再按值归类；
3. 最后按值做查询或分类讨论。

这是一种很常见的“二次抽象”。

七、第六层：为什么查询可以离线双指针

题目里的查询 k 是有序的。

而每个值 x 在某次查询里属于哪一类，只取决于：

- x 和某个阈值的大小关系。

这就说明每个查询并不需要从头重新算一遍。

更聪明的做法是：

1. 把不同的值按大小排序；
2. 用一个指针维护当前已经“进入阈值范围”的值；
3. 随着查询增大，只增量更新统计量。

这就是很标准的：

- 离线；
- 双指针；
- 扫描阈值。

以后你看到：

- 查询有序；
- 分类只和某个阈值比较有关；

就要想到这个套路。

八、第七层：为什么这里会出现逆元和快速幂

因为题目中需要在模意义下处理：

$$\frac{1}{a_i} \pmod{mod}$$

当模数是质数时，可以把它写成：

$$a_i^{mod-2} \pmod{mod}$$

所以这类题自然会带出：

- 快速幂；
- 逆元；
- 模意义分数。

这一步本身不难，但在综合题里往往是必经之路。

九、这题最值得带走的四件事

1. 贪心题要学会比较边际收益

这是全题灵魂。

2. 一旦答案只和极值有关，就想到单调栈

这是结构转化的关键。

3. 单调栈统计完以后，不一定就结束了

可能还要按值继续合并。

4. 查询有序时，优先想离线扫描

别每个查询都重新做。

十、易错点

1. 没有真正证明“所有操作都给最小值”。
2. 单调栈的严格 / 非严格边界写错。
3. 只按位置统计，没有继续按值合并。
4. 离线阈值边界写错。
5. 模意义下减法忘记补 mod 。

G. Grid Path

一、这题为什么有代表性

这题是非常典型的“后段题思维训练”。

它表面上是个网格路径问题，但正解不是图搜索，也不是简单 DP。

真正的解题过程是：

1. 把网格中的访问集合抽象成每一行的区间；
2. 把相邻行之间的关系抽象成区间相交；
3. 再把二维区间状态压成少量统计量；
4. 最后把大步数线性转移交给矩阵快速幂。

这整套过程非常值得学习，因为它训练的是高层抽象能力。

二、第一步：为什么每一行访问到的格子一定是连续区间

这一步是全题的起点。

因为在同一行，你只能：

- 左移；

- 右移。

所以如果某一行访问到了两格：

- $(i, 1)$
- (i, r) ，其中 $1 < r$

那么中间所有格子一定也会被经过。

于是这一行访问到的格子集合一定是一个连续区间：

$$[l_i, r_i]$$

这一步看起来简单，但非常关键。

以后你看到类似的网格题时，只要某一维上的移动方式是连续的，就要主动去检查：

- 访问集合是不是天然形成一个区间。

这往往就是状态压缩的入口。

三、第二步：为什么相邻两行区间必须相交

因为从上一行到下一行，只能往下走。

也就是说，必须存在某一列，使得你可以从上一行落到下一行。

于是：

- 第 i 行访问区间；
- 第 $i+1$ 行访问区间；

必须有交集。

这一步非常重要，因为它把原来的网格路径问题，直接转化成了：

数一数有多少个区间序列，满足相邻区间相交。

一旦你把题目改写成这个形式，后面就不再是网格题，而是区间 DP 题了。

四、第三步：为什么不能老老实实设 $dp[l][r]$

当然可以设：

- $dp[l][r]$ 表示当前层区间为 $[l, r]$ 的方案数。

但问题在于：

- 区间个数是 $O(m^2)$ ；
- 转移时还要考虑所有与它相交的旧区间；
- 复杂度会非常难看。

所以这题真正考你的，是下一步：

新区间 $[x, y]$ 到底依赖旧状态的什么信息？

只要这个问题答出来，状态压缩就有了希望。

五、第四步：为什么“统计相交”要改成“总和减不相交”

如果你直接数“和 $[x, y]$ 相交的区间”，会很麻烦。

但反过来想，不相交的区间只有两类：

1. 完全在左边： $r < x$
2. 完全在右边： $l > y$

于是，

- 相交区间的贡献和
= 全部区间贡献和
 - 左边完全分离的贡献和
 - 右边完全分离的贡献和

这一步特别值得记。

因为它是一类非常常见的技巧：

直接统计“满足条件”的对象很难时，
先统计“全部”，再减去“不满足条件”的部分。

这种“补集思维”以后会在很多区间题、图论题、计数题里反复出现。

六、第五步：为什么能够压缩状态

现在你会发现，新区间 $[x, y]$ 的值，并不需要知道旧的每一个 $dp[l][r]$ 。

它真正依赖的是：

- 所有旧状态总和；
- 某些关于右端点的聚合量；
- 某些关于左端点的聚合量。

也就是说，下一步根本不关心旧状态的完整细节，只关心这些统计量。

这就是状态压缩最经典的判断标准：

如果下一步只依赖少量聚合量，那么原状态就可以被这些聚合量替代。

这句话非常重要。

以后你做状态压缩题时，建议你反复问自己：

- “下一步真的需要知道所有细节吗？”
- “还是说，只需要知道若干统计量就够了？”

很多时候，正解就藏在这里。

七、第六步：为什么最后自然会想到矩阵快速幂

经过压缩之后：

- 状态维度不大；
- 下一步状态是旧状态的线性组合；
- 但步数 n 很大。

这三个条件同时出现时，第一反应就应该是：

- 矩阵快速幂。

因为线性转移可以写成：

$$f_{t+1} = M \cdot f_t$$

所以：

$$f_t = M^{t-1} \cdot f_1$$

这就是标准模型。

这题最值得训练你的地方之一，就是：

看到“大步数 + 小维线性转移”时，要足够敏感地想到矩阵快速幂。

八、第七步：为什么这题还会卡常

很多同学以为：

- 理论复杂度对了，就一定能过。

但这题恰好告诉你，不一定。

因为矩阵快速幂虽然可做，但常数不小。

所以还需要实战优化。

例如：

1. 不要先完整求出 M^k

如果最终只需要：

$$M^k \cdot f$$

那么在快速幂过程中，当前二进制位为 1 时，直接把矩阵乘到向量上即可。

这样能少做一部分不必要的矩阵乘法。

2. 右矩阵转置

矩阵乘法中先把右矩阵转置，可以明显改善缓存命中，常数会好很多。

3. 减少取模次数

内层不要每次乘法都 `% mod`，可以先累计一段，再统一处理。
当然前提是你要控制好溢出。

这类优化虽然不改变算法级别，但在实战里经常决定生死。

九、这题最值得带走的四件事

1. 网格题不一定按格子做

很多网格题真正应该维护的，不是具体格子，而是：

- 一行的区间；
- 一列的轮廓；
- 一个边界形状。

2. 相交关系常常可以转补集

这是非常高频的技巧。

3. 状态压缩时，要盯住“下一步依赖什么”

不是盯住原状态本身。

4. 大步数线性转移，一定要想到矩阵快速幂

这是后段题常见模型。

十、易错点

1. 没有证明“每一行是区间”。
 2. 没有看出“相交”可以改成“不相交的补集”。
 3. 状态压缩不彻底，仍然维护了太多细节。
 4. 矩阵快速幂实现常数过大。
-

本场比赛的知识点归并

为了以后复习时更有结构，这里把本场题按专题重新归并一次。

一、观察与规律提炼

涉及题目：

- A. Passing the Ball
- B. Right Maximum

核心训练：

- 不盲目模拟；

- 找第一个关键位置；
- 看清删完或走完后剩下的结构。

二、周期统计与容斥

涉及题目：

- C. Spring

核心训练：

- 倍数个数；
- gcd / lcm；
- 按贡献理解容斥，而不是死背公式。

三、图论建模与二分图

涉及题目：

- D. Alternating Path

核心训练：

- 从“交替性”联想到二分图；
- 连通块独立处理；
- BFS/DFS 染色。

四、构造与频数匹配

涉及题目：

- E. Sum of Digits (and Again)

核心训练：

- 数值过程与字符过程分离；
- 先定后缀，再补前缀；
- 特判与前导零意识。

五、综合题：贪心 + 单调栈 + 离线

涉及题目：

- F. Sum of Fractions

核心训练：

- 比较边际收益；
- 把问题转成极值贡献；
- 单调栈统计；

- 按值合并；
- 离线扫描。

六、状态压缩与矩阵快速幂

涉及题目：

- G. Grid Path

核心训练：

- 网格转区间模型；
- 相交转补集；
- 聚合量状态压缩；
- 大步数线性转移。

本场比赛最值得反复复习的三题

如果以后时间不够，不一定每题都完整重做。
但下面三题非常值得反复吸收。

1. D 题

因为它训练你把题意语言翻译成模型。
这类能力在图论题里非常关键。

2. F 题

因为它把很多高频思想揉在一起：

- 边际收益贪心；
- 子数组最小值贡献；
- 单调栈；
- 离线；
- 逆元。

它是本场最典型的综合题。

3. G 题

因为它训练的是更高层的抽象能力：

- 从网格过程提炼区间状态；
- 从区间状态提炼聚合量；
- 再转成矩阵快速幂。

这类能力一旦掌握，会明显提高你做后段题的稳定性。

赛后真正应该带走的东西

如果把这场比赛压缩成几句话，我希望你最后真正带走的是下面这些东西。

1. 过程题不一定要模拟

如果题目给了很多操作，不要被它牵着走。

先问自己：

- 有没有一个关键位置？
- 有没有一个关键极值？
- 有没有一个关键统计量？

2. 综合题要学会先拆小单位

先研究单个对象，再看多个对象之间怎么互动。

这比一上来想整体稳得多。

3. 一旦答案只和极值有关，就想到单调栈

这是很高频的桥梁。

4. 查询有序时，想到离线扫描

不要每个查询都重头算。

5. 大步数的线性转移，想到矩阵快速幂

这要逐渐形成反射。

复习建议

如果你以后回头复习这场，推荐按下面顺序来。

第一轮：重看“题目本质”

只看每题的核心观察，不急着看实现。

重点问自己：

- 这题真正决定答案的是什么？
- 我当时为什么没想到？

第二轮：按专题复习

重点复习以下三个专题：

1. 二分图建模；
2. 子数组极值贡献与单调栈；
3. 状态压缩与矩阵快速幂。

第三轮：重新回答三个问题

对每道题重新回答：

1. 我为什么会想到这个方向？
2. 这题最关键的结论是什么？
3. 以后什么题里我应该想到这个套路？

如果这三个问题你都能答出来，这场比赛才算真正吸收了。

结语

`Educational Codeforces Round 188` 这场很适合拿来做一次完整的“思维训练”。

它不是一场纯模板场，也不是一场纯细节场。

它更像一场能力训练课：

- 前面教你不要乱模拟；
- 中间教你把题面翻译成模型；
- 后面教你把复杂状态压成核心统计量。

如果你能把这场真正吃透，那么以后在很多中档题、综合题里，你的思路都会更稳，建模也会更快。

这才是一场比赛总结最有价值的地方。